SOFTWARE REQUIREMENTS SPECIFICATION

for

Boomaga-IPP

Version 0.2.1 (Revised Draft Baseline)

Prepared by Gary S. Martin
with AI tool assistance

***Boomaga-IPP* Community**

July 10, 2026

Revision History

Revision	Date	Author(s)	Description
0.1.0	2026-06-10	Gary S. Martin	Draft Baseline
0.2.0	2026-07-08	Gary S. Martin	Revised Draft Baseline: Corrected Issues 1, 3-7, 9, & 11-12
0.2.1	2026-07-10	Gary S. Martin	Added UML Source Listings to Appendix C.

Legal and Regulatory Requirements

Adobe® is a registered trademark of Adobe, Inc.

ARCH LINUX™ is a recognized trademark owned by Levente Polyák and Judd Vinet on behalf of the *Arch Linux Project*.

Cargo® is a registered trademark of The Rust Foundation.

CLAUDE® is a registered trademark of Anthropic PBC.

GhostScript® is a registered trademark of Artifex Software, Inc.

GitHub® is a registered trademark of GitHub, Inc., a subsidiary of Microsoft Corporation.

IPP Everywhere™ is a recognized trademark owned by the Institute of Electrical and Electronic Engineers (IEEE) Industry Standards and Technology Organization (ISTO). The IEEE ISTO filed an application for the use of *IPP Everywhere*™ as a registered trademark in the U.S. on January 15, 2014, and abandoned this application as of September 11, 2017.

Linux® is a registered trademark of Linus Torvalds in the U.S. and other countries.

LLAMA® is a registered trademark of Meta Platforms, Inc.

opencode® is a registered trademark of Anomaly Innovations Inc.

POSTSCRIPT® is a registered trademark of Adobe, Inc.

RUST® is a registered trademark of The Rust Foundation.

UNIX® is a registered trademark of The Open Group.

Contents

List of Figures

List of Tables

1. Introduction

1.1. Purpose

The purpose of the *Boomaga-IPP Virtual Printer* system is to replace the venerable, but now orphaned *Boomaga*[1] virtual printer (Sokolov et al., 2013b; Sokolov et al., 2013a) with a free, open-source, memory-safe, and type-safe virtual printer system for current *Linux*® computer systems. The *Boomaga-IPP Virtual Printer* system is intended to conform to the current standards of the IEEE ISTO Printer Working Group (PWG): Internet Printing Protocol (IPP) Version 2.0 (IPP/2.0) and *IPP Everywhere*™. *Boomaga-IPP* is targeted at *systemd*-managed *Linux*® systems with *Wayland* display environments. It is to be written entirely in the *RUST*® programming language and is to use the *Xilem GUI Framework*.

1.2. Document Conventions

The format of this document is adapted from the *LaTeX template for IEEE Software Requirements Specifications* (Eisenbarth, 2014-09-06/2026). M. Eisenbarth's template is partially derived from the code of Yiannis Lazarides and the template of Karl E. Wiegers. The document template is based upon and employs the document conventions of *IEEE 830-1998, Recommended Practice for Software Requirements Specifications* (IEEE Computer Society, 1998). *IEEE 830-1998* is a standard of the IEEE Computer Society, and is not freely available. While it was used in the development of the *LaTeX template*, it was not referenced directly in the production of this document.

The use of the word "shall" in this document connotes a requirement. Each requirement herein is numbered as a Level 1 (*L1*) *SYS* Requirement and is sequentially numbered based on its order of occurrence in the initial draft of the document. To ensure traceability, requirements will not be renumbered if the document changes. Any requirements that are deleted will result in gaps in the numbering and any new requirements added will have higher numbers than previously numbered requirements regardless of their place in the specification or their precedence.

Within this software requirements specification (SRS), the priorities of higher-level parent requirements are assumed to be inherited by their detailed child requirements unless otherwise specified. In the event of any discrepancy between this SRS and any lower level *Boomaga-IPP* specifications, the SRS shall be the governing document.*(L1-SYS-001)*

1.3. Intended Audience and Reading Suggestions

This SRS is intended for use by any and all members of the *Boomaga-IPP* Project Team, including the project manager, software developers, alpha and beta testers, and documentation writers. This

[1] Boomaga is a portmanteau of "BOOklet MAnager"

SRS establishes all functional and nonfunctional system-level (Level 1) requirements governing the development of the *Boomaga-IPP Virtual Printer* system. Any and All *Boomaga-IPP* team members should familiarize themselves with the entirety of this specification. Technically inclined *Boomaga-IPP* users may also wish to familiarize themselves with any sections of this SRS that interest them.

1.4. Project Scope

The objective of the *Boomaga-IPP* project is to deliver a fully-functional, easily-installable, free, and open-source virtual printer system for current *Linux*® computer systems. This virtual printer system is intended to provide the same or greater functionality as the original *Boomaga* virtual printer system with greater reliability and the same or greater ease of use. This virtual printer system shall be comprised of both software and documentation.*(L1-SYS-002)* The *Boomaga-IPP* project team intends to maintain the software and documentation for at least five years from initial release or until the system is replaced by a virtual printer system providing the same or greater functionality for the majority of common *Linux*® desktop environments.

1.5. References

Adobe, Inc. (nodate). *PostScript* (Version 3). Retrieved 2026-05-13, from https://www.adobe.com/products/postscript.html

Anomaly Innovations Inc. (nodate). *OpenCode*. Retrieved 2026-05-13, from https://opencode.ai/

Anthropic PBC. (nodate-a). *Claude* (Version Opus 4.7). Retrieved 2026-05-12, from https://claude.com/product/overview

Anthropic PBC. (nodate-b). *Claude Code* (Version Opus 4.7). Retrieved 2026-05-12, from https://claude.com/product/claude-code

Artifex Software, Inc. (nodate). *Ghostscript*. Retrieved 2026-05-12, from https://www.ghostscript.com/index.html

Berkenbilt, J., & Holger, M. (nodate). *QPDF*. Retrieved 2026-05-13, from https://qpdf.sourceforge.io/

Bruno, L., & Khan, Z. A. (2026-06-26). *Zbus_systemd library crate* (Version 0.26100.0). crates.io. Retrieved 2026-05-25, from https://crates.io/crates/zbus_systemd

Cedilnik, A., Hoffman, B., King, B., Martin, K., & Neundorf, A. (2026-05-21). *CMake* (Version 4.3.3). Retrieved 2026-06-10, from https://cmake.org/

Dröge, S., Wenger, A., Elmoussaoui, B., & Maximiliano. (2026-02-21). *Poppler-rs* (Version 0.26.0). crates.io. Retrieved 2026-04-22, from https://crates.io/crates/poppler-rs

Eisenbarth, J.-P. (2026-01-20). *SRS-Tex: A LaTeX template for ISO/IEEE Software Requirements Specifications*. Retrieved 2026-01-26, from https://github.com/jpeisenbarth/SRS-Tex

Gettys, J., Scheifler, B., & The X.org Project. (nodate). *X.Org Stack* (Version X11R7.7). Retrieved 2026-05-13, from https://x.org/wiki/

GitHub, Inc. (A subsidiary of Microsoft Corporation). (2026). *GitHub*. GitHub. Retrieved 2026-05-12, from https://github.com/

GNU Project & Smith, P. (2026-04-12). *GNU Make* (Version 4.4.1). Retrieved 2026-06-10, from https://www.gnu.org/software/make/

Høgsberg, K., & The Poppler Project. (nodate). *Poppler* (Version 26.05.0). Retrieved 2026-03-10, from https://poppler.freedesktop.org/

Høgsberg, K., & The Wayland Project. (nodate). *Wayland*. Retrieved 2026-05-13, from https://wayland.freedesktop.org/
IEEE Computer Society. (1998-10-20). *IEEE Recommended Practice for Software Requirements Specifications | IEEE 830-1998*. Retrieved 2026-02-08, from https://store.accuristech.com/standards/ieee-830-1998?vendor_id=1222&product_id=14024
ISO/IEC JTC 1/SC 22/WG 9 - Ada. (2023-05). *ISO/IEC 8652:2023*. Retrieved 2026-05-12, from https://www.iso.org/standard/83621.html
ISO/TC 171/SC 2. (2008). *ISO 32000-1:2008* (2008th ed.). Retrieved 2026-06-10, from https://www.iso.org/standard/51502.html
KDE Community. (nodate). *Plasma Desktop* (Version 6). Retrieved 2026-05-13, from https://kde.org/plasma-desktop/
KDE Community. (2026-05-14). *KDE Community*. KDE Community. Retrieved 2026-05-12, from https://kde.org/
Levien, R., Rofls, C., & Kuut, K. (2023-02-28). *Druid* (Version 0.8.3). Retrieved 2026-05-12, from https://crates.io/crates/druid
Linebender Organization. (nodate-a). *Linebender*. Linebender. Retrieved 2026-03-11, from https://linebender.org/
Linebender Organization. (nodate-b). *Linebender Xilem Development*. Xilem. Retrieved 2026-04-02, from https://www.xilem.dev/
Linebender Organization. (2026-03-08). *Xilem Library Crate*. crates.io. Retrieved 2026-03-10, from https://crates.io/crates/xilem
Linus Torvalds, Linux Kernel Organization PBC, & Linux Foundation. (nodate). *Linux Kernel* (Version 7.0). Retrieved 2026-05-12, from https://www.kernel.org/
Linux Foundation. (nodate). *Linux Foundation*. Linux Foundation. Retrieved 2026-05-13, from https://www.linuxfoundation.org
Martin, G. S., & Boomaga-IPP Project Team. (2026a-03-06). *Boomaga-IPP Project Repository*. github.com. Retrieved 2026-03-11, from https://github.com/GaryScottMartin/Boomaga-IPP
Martin, G. S., & Boomaga-IPP Project Team. (2026b-07-08). *Boomaga-IPP User Interface Specification* (latest). Retrieved 2026-07-08, from https://github.com/GaryScottMartin/Boomaga-IPP/blob/2721b4d4d404a924ecb01f4cc4017cfc96b8a4ec/docs/User-Interface-Spec--latest.pdf
Meta AI. (nodate). *Llama Family of AI Models*. Retrieved 2026-05-15, from https://www.llama.com/
Pankratov, D. (2026-02-26). *Qpdf-rs* (Version 0.3.5). crates.io. Retrieved 2026-04-30, from https://crates.io/crates/qpdf
Pennington, H., Larsson, A., Carlsson, A., & D-Bus Project. (nodate). *D-Bus*. Retrieved 2026-05-12, from https://www.freedesktop.org/wiki/Software/dbus/
Poettering, L., & Sievers, K. (nodate). *System and Service Manager (systemd)*. Retrieved 2026-05-12, from https://systemd.io/
Printer Working Group. (2020-05-15). *IPP Everywhere™*. Retrieved 2026-05-11, from https://ftp.pwg.org/pub/pwg/candidates/cs-ippeve11-20200515-5100.14.pdf
Printer Working Group. (2024-11-08). *Internet Printing Protocol/2.x* (Fourth). https://ftp.pwg.org/pub/pwg/standards/std-ippbase23-20241108-5100.12.pdf
Roques, A., & PlantUML Contributors. (2026-05). *PlantUML*. Retrieved 2026-05-15, from https://plantuml.com/
Rust Foundation. (nodate-a). *The Cargo Book*. The Cargo Book. Retrieved 2026-05-12, from https://doc.rust-lang.org/cargo/

Rust Foundation. (nodate-b). *The Rust Package Registry (crates.io)*. The Rust community's crate registry. Retrieved 2026-03-11, from https://crates.io/

Rust Foundation. (nodate-c). *The Rust Programming Language*. The Rust Programming Language. Retrieved 2026-05-13, from https://doc.rust-lang.org/stable/book/

Sokolov, A., famo, Ip, C., Moskvin, A., Devendra, Pérez Meyer, L. D. N., Borisenko, L., Ekhlakov, O., SDDM, Inflianskas, R., & Kernozhitsky, A. (2013a-01-26). *Boomaga*. Retrieved 2026-02-08, from https://github.com/Boomaga/boomaga

Sokolov, A., famo, Ip, C., Moskvin, A., Devendra, Pérez Meyer, L. D. N., Borisenko, L., Ekhlakov, O., SDDM, Inflianskas, R., & Kernozhitsky, A. (2013b-01-26). *Boomaga - Virtual Printer - Project Web Site*. Retrieved 2026-01-26, from https://www.boomaga.org/

Srinivas, A., Yarats, D., Ho, J., Konwinski, A., & Perplexity Ai, Inc. (nodate). *Perplexity.ai*. Retrieved 2026-05-15, from https://www.perplexity.ai/

Sweet, M. R., & OpenPrinting. (nodate). *OpenPrinting CUPS*. Retrieved 2026-05-12, from https://openprinting.github.io/cups/

The Open Group. (nodate). *UNIX®*. Retrieved 2026-05-13, from https://www.unix.org/

Torvalds, L., Hamano, J., & Git Community. (nodate). *Git*. Retrieved 2026-05-23, from https://git-scm.com/

Vinet, J., Griffin, A., Polyák, L., Carlier, L., Fleischer, L., Gauduin, M., Gregory, A., Haase, S.-H., Hesse, C., Hötzel, J., Linderud, M., Löthberg, J., Luttringer, S., McRae, A., Pomozov, A., Powalowski, T., Radke, A., Razzolini, G., Rojas, A., ... Yan, F. (nodate). *Arch Linux*. Retrieved 2026-05-12, from https://archlinux.org/

2. System Concept[1]

2.1. Product Perspective

Boomaga is a virtual printer for viewing a document before printing it out to either a physical printer or another virtual printer on *Linux*®-based computer systems. It is also capable of printing booklets and other arrangements of multiples pages per sheet, and of combining multiple documents into a single print job. *Boomaga* is an open source software project mainly distributed under the GPLv2 license, although some files are distributed under the LGPLv2+ license. The *Boomaga* project began in 2012 and the first software release was Dec 14, 2013. The final release of *Boomaga*, version 3.0.0, was on March 13, 2019. On February 20, 2022, a patch to provide compatibility with *GhostScript*® V9.54 was merged with the *Boomaga* source code, but no release was issued. *Boomaga* has been unmaintained since that date.

Basic compatibility with *Wayland* was added to *Boomaga* in v3.0.0 in 2019, but *Boomaga* was primarily designed to work under *X11* graphical environments (which either have been or are actively being replaced by *Wayland* environments in popular *Linux*® distributions). In addition, the *D-Bus* mechanism used by *Boomaga* for communication between its user-space GUI and the host computer's root-level *CUPS* print driver has proven unreliable for many users, preventing the GUI from automatically starting when the user prints a file to *Boomaga*. A number of recent minor issues have also arisen from incompatibilities between *Boomaga* and regular version updates in the tool chain and library packages used to build and install it.

Boomaga-IPP was conceived in late 2025 as a replacement for *Boomaga*. Development of *Boomaga-IPP* began in February 2026 with the assistance of newly available artificial intelligence coding tools including *CLAUDE*® *Code*, *opencode*®, *Perplexity.ai*, and several open-source, locally-run large language models (LLMs) developed for coding.

Boomaga-IPP will not reuse any *Boomaga* code, but is to be written from scratch in a high-reliability language to provide type safety and memory safety. *Ada* was briefly considered (based on the project manager's experience with it), but was rejected due the lack of existing *Ada* language bindings to the libraries needed by *Boomaga-IPP*. It was decided that *Boomaga-IPP* will be written in the *RUST*® programming language, as the needed *RUST*® language bindings are readily available.

Boomaga-IPP is targeted at *Linux*® systems employing *Wayland* display architectures and will use an *IPP Everywhere*™ client in place of a printer driver. *Boomaga-IPP* is intended to provide the same or greater functionality as the original *Boomaga*.

Boomaga-IPP will support the *IPP Everywhere*™ mandatory document formats: PDF (application/pdf), PWG Raster (image/pwg-raster), and JPEG (image/jpeg). Since these do not include *POSTSCRIPT*®, *Boomaga-IPP* will not continue *Boomaga*'s support of *POSTSCRIPT*®. Consequently, *Boomaga-IPP* will use the combination of *Poppler* and *qpdf* to replace the remaining required functions performed by *GhostScript*® in *Boomaga*.

[1] For the sake of completeness of the documents, this chapter is included as the second chapter of both the Software Requirements Specification and the User Interface Specification.

2.2. Product Functions

The top-level functional requirements of the *Boomaga-IPP Virtual Printer* system are allocated to the following major software components:

***Boomaga-IPP* Capture_Daemon** A daemon that captures input print jobs through an *IPP Everywhere*™ print service and adds them to the *Boomaga-IPP* Job_Store.

***Boomaga-IPP* GUI** A desktop application launched by and operating in cooperation with the *Boomaga-IPP* Imposition_Engine. The *Boomaga-IPP GUI Application* allows the user to select the desired imposition settings, preview the output PDF (print job), select the desired downstream printer, select the desired downstream printer settings, and request the printing of the output print job.

***Boomaga-IPP* Imposition_Engine** A daemon that constructs the output print job by imposing the input print job(s) according to the imposition settings and document page manipulation requests received from the user through the *Boomaga-IPP GUI Application*. The Imposition_Engine also sends the output print job to the GUI Job_Store_PDF_Preview feature and the *Boomaga-IPP* Downstream_Printer in response to *Boomaga-IPP Virtual Printer* requests.

***Boomaga-IPP* Downstream_Printer** A daemon that manages the interface between the *Boomaga-IPP Virtual Printer* and the host computer's *CUPS* server, enabling the *Boomaga-IPP* user, through the *Boomaga-IPP GUI Application*, to select a *CUPS*-backed printer, change printer settings of the selected printer, and submit the output print job to the selected downstream printer. The downstream *CUPS* printer may be a physical printer or another virtual printer such as *CUPS-PDF*.

A Unified Modeling Language (UML) component diagram of these software components is displayed below in Figure 2.1. Additionally, the UML component diagram, class diagram, sequence diagram, and use case diagram for the *Boomaga-IPP Virtual Printer* system are included in Appendix C, Analysis Models, as Figures C.1, C.2, C.3, and C.4, respectively. The UML source code for all these diagrams are also in Appendix C as Tables C.1, *Boomaga-IPP Virtual Printer* System Component UML Model Source; C.2, *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 1; C.3, *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 2; C.4, *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 3; C.5, *Boomaga-IPP Virtual Printer* System Sequence UML Model Source; and C.6, *Boomaga-IPP Virtual Printer* Use Case UML Model Source.

1. The *Boomaga-IPP* Capture_Daemon will allow *Boomaga-IPP Virtual Printer* users to print through the host computer's *CUPS* server to the *Boomaga-IPP Virtual Printer* via a provided *IPP Everywhere*™ client. The *IPP Everywhere*™ client will:
    a) Interface through *CUPS*,
    b) Queue the printed file to the virtual printer,
    c) Provide document merging and batch printing, and
    d) Launch the *Boomaga-IPP* GUI application on the user's desktop.

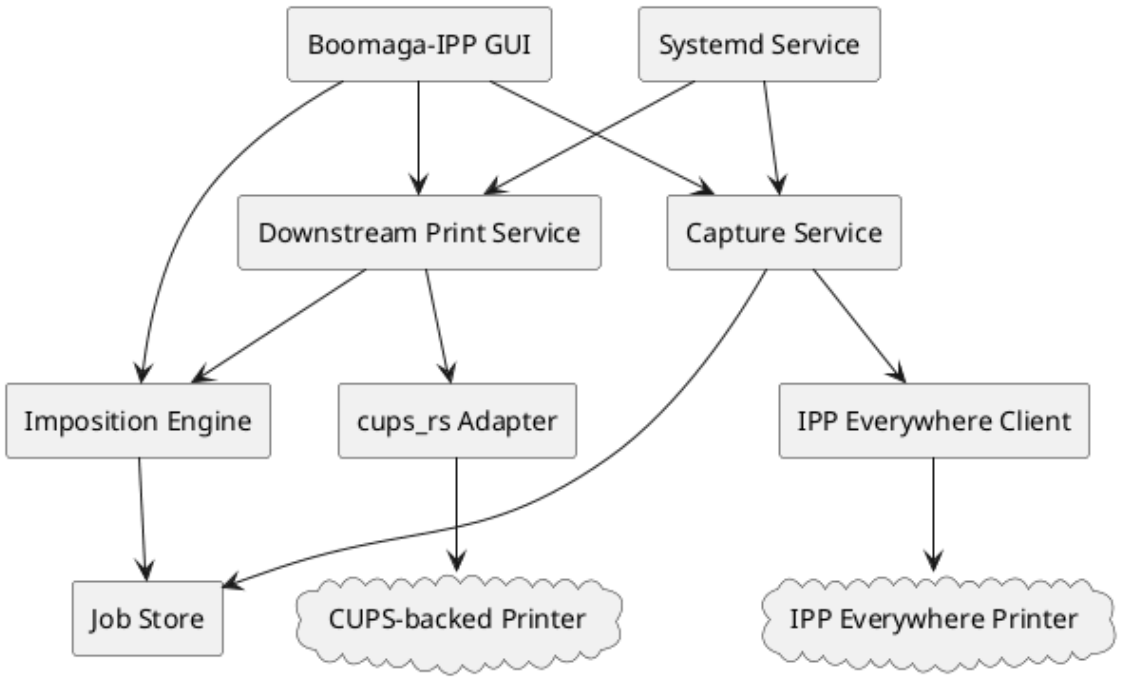

Figure 2.1.: *Boomaga-IPP Virtual Printer* System Component Diagram.

2. The *Boomaga-IPP Virtual Printer* will present a GUI to the user once an input job is received by the Capture_Daemon and added to the Imposition_Engine Job_Store. The *Boomaga-IPP GUI Application* will:
    a) Preview prints to the user prior to physical printing;
    b) Allow the user to select any other *CUPS* printer for the virtual printer output;
    c) Allow the user to print booklets with automatic page arrangement;
    d) Allow the user to print multiple pages per sheet (1, 2, 4, 8 pages per sheet) with horizontal and vertical page ordering;
    e) Allow the user to manipulate pages by rotating pages, deleting pages, adding blank pages, and reordering pages;
    f) Allow the user to create and manage sub-booklets;
    g) Provide duplex printing for both duplex-capable and manual duplex printers; and
    h) Send the virtual printer output to the downstream *CUPS* printer in the form selected by the user via the *Boomaga-IPP* Downstream_Printer daemon.
3. The *Boomaga-IPP Virtual Printer* will provide inter-process communication between the *Boomaga-IPP* software components described above as required to execute the functions of the *Boomaga-IPP Virtual Printer*, including printing the user's desired output to the downstream *CUPS* printer.

2.3. User Classes and Characteristics

A single class of *Boomaga-IPP Virtual Printer* users is envisioned. All users are expected to install the *Boomaga-IPP Virtual Printer* using the provided package manager of their *Linux*® distribution whenever it is available through that channel. After installation, users will employ *Boomaga-IPP* by selecting it from the menu provided by *CUPS* to the various applications installed on the users' systems and print a single file or a series of files to the *Boomaga-IPP Virtual Printer*. Printing a file to the *Boomaga-IPP Virtual Printer* will cause the *Boomaga-IPP GUI Application* to launch automatically on the user's desktop. Using the *Boomaga-IPP GUI Application*, users will be able to:

1. Manipulate pages as desired,
2. Select the desired imposition format,
3. Select the downstream *CUPS* printer and change its settings, and
4. Print the output to the selected downstream printer.

The *Boomaga-IPP* user experience is expected to be very similar to the experience currently provided by *Boomaga*.

2.4. Operating Environment

Boomaga-IPP's initial development will take place on an x86-64 computer system running current versions of the *ARCH LINUX*™ distribution, *Wayland*, and *KDE Plasma*. The x86-64 architecture is the primary target architecture for the *Boomaga-IPP Virtual Printer* system. The software environment necessary to support *Boomaga-IPP* is identified in Table 2.1 (including the versions in use as of the date of this document). It is expected that any ARM-64 system capable of hosting a robust modern *Linux*® distribution and *Wayland* display environment should also be capable of compiling the *Boomaga-IPP* system. The *Boomaga-IPP* team intends to test the system on an ARM-64 architecture computer system at a later time.

2.5. Design and Implementation Constraints

All *Boomaga-IPP* software will be released under the GNU General Public License, version 3 (GPLv3). All *Boomaga-IPP* software will be written in the *RUST*® programming language. The *Boomaga-IPP Virtual Printer* system will be initially targeted at modern x86-64 computer systems running a *Linux*® version 7 or later kernel, the *Wayland* display architecture, and *systemd* for system administration. The *Boomaga-IPP* GUI will be based on the *Linebender Organization Xilem* GUI toolkit (Linebender Organization, nodate-a, nodate-b, 2022-11-16/2026). *Boomaga-IPP* will use *Poppler-rs* (Dröge et al., 2026; Høgsberg & The Poppler Project, nodate) for document rendering. *Boomaga-IPP* will use the *zbus_systemd* (Bruno & Khan, 2026) library for inter-process communication (IPC) with *systemd*.

Table 2.1.: *Boomaga-IPP* Software Environment.

Core System Software		Rust crates			
Required Software	Version	Required Software	Version	Required Software	Version
Linux kernel	6.18.16	GUI		Logging	
glibc	2.43	xilem	0.4	tracing	0.1
Wayland	1.24.0	kurbo	0.11	tracing-subscriber	0.3
systemd	259.3	winit	0.3		
poppler-glib	26.0.2	cairo-rs	0.18		
qpdf	12.3.2	Document Rendering		Document Manipulation	
		poppler-rs	0.26.0	qpdf	0.3.5
		libc	0.2		
		IPC		Configuration	
		zbus_systemd	0.26100.0	config	0.14
				directories	5.0
		Async Runtime		Time	
		tokio	1.35	chrono	0.4
		async-trait	0.1	URL	
				url	2.5
		Serialization		Temporary Files	
		serde	1.0	tempfile	3.1
		serde_json	1.0		
		Error Handling		UUID	
		thiserror	1.0	uuid	1.0
		anyhow	1.0		

2.6. User Documentation

The primary user documentation will be the README.md file on the *Boomaga-IPP* github archive (Martin & Boomaga-IPP Project Team, 2026-03-03/2026a). At a later time, a frequently asked questions (FAQ) page will be added to the wiki on the *Boomaga-IPP* github archive. Any supplemental documentation developed will be hosted there, as well.

2.7. Assumptions and Dependencies

Successful implementation of *Boomaga-IPP* relies upon two immature technologies: artificial intelligence (AI) coding assistance and the *Linebender Xilem* GUI toolkit. The project initiator had virtually no knowledge of the *RUST*® programming language at the start of the project. As a result, a heavy reliance is being placed on AI assistance. *CLAUDE*® *Code*, *opencode*®, and *Perplexity.ai* were used to establish the initial project requirements and system architecture. Cost limitations soon dictated the use of the *GLM-4.7-flash* local LLM together with *CLAUDE*® *Code* to author the prototype code. The code was further developed with the *Meta-Llama-3.1-8B-Instruct* model because of its superior speed. It is expected that additional AI tools will be used as they emerge and mature. A number of local and cloud-based options are being assessed for code review and testing. The maturity and cost of the various options will have a substantial impact on the quality and functionality of the final code and may well determine whether or not the *Boomaga-IPP* project produces a usable virtual printer implementation.

The selection of the *Linebender Organization*'s *Xilem* GUI toolkit for use in the development of the *Boomaga-IPP* GUI was made because it was judged to be the best option for producing a maintainable *RUST*® based system over the long term. The original choice was the *Linebender Druid* toolkit, also written entirely in *RUST*®. However, *Druid* is deprecated and no longer under development by the *Linebender Organization*. As one of the goals of this project is to produce a virtual printer system that is maintainable over a decade or more, it was decided to move from *Druid* to *Xilem* even though *Xilem* is alpha level code as of the date of this document. Consequently, the maintainability of the *Boomaga-IPP* system depends upon the *Linebender Organization* continuing its development of the *Xilem* toolkit through beta testing and into a stable production framework.

3. External Interface Requirements

3.1. Interfaces to the User

3.1.1. Installation of *Boomaga-IPP* on a *Linux®* host computer

The intended installation interface between the *Boomaga-IPP Virtual Printer* and its users shall be installation packages specific to each of the well-known *Linux®* distributions.*(L1-SYS-003)* Wherever possible, these shall be provided through the distribution's preferred installation package repository.*(L1-SYS-004)* Instructions for user installation directly from the *Boomaga-IPP GitHub Repository* (Martin & Boomaga-IPP Project Team, 2026-03-03/2026a) shall also be provided.*(L1-SYS-005)*

Host Distribution Software Repository Interface

The *Boomaga-IPP Virtual Printer* shall be packaged to interface with the common application installation tools and methods provided by common, widely-deployed *Linux®* distributions (e.g., rpm, deb, and pkg.tar.zst) and made available to package maintainers on the respective distributions.*(L1-SYS-006)*

Boomaga-IPP Project Repository Installation Interface

For instances when a standard format installation package for a given *Linux®* distribution is not available, generic step-by-step instructions shall be provided to allow users to manually install the *Boomaga-IPP Virtual Printer* software system directly from *Boomaga-IPP Project Repository* archive. These instructions shall adequately describe all actions required to install the *Boomaga-IPP Virtual Printer* from the *Boomaga-IPP Project Repository* as listed in Table 3.1.*(L1-SYS-007)*

Table 3.1.: *Boomaga-IPP Project Repository* Required Installation Actions

1. Create any necessary directories,
2. Clone the *Boomaga-IPP Project Repository* (Martin & Boomaga-IPP Project Team, 2026-03-03/2026a) to the host *Linux®* computer,
3. Compile, link, and install the *Boomaga-IPP Virtual Printer* software components using *CMake* and *GNU Make*,
4. Configure the various *Boomaga-IPP Virtual Printer* services with *systemd*, and
5. Test the resulting *Boomaga-IPP Virtual Printer* installation.

(L1-SYS-007)

3.1.2. The *Boomaga-IPP* Graphical User Interface

The sole operational interface between the *Boomaga-IPP Virtual Printer* and its user shall be the *Boomaga-IPP GUI Application*.*(L1-SYS-008)* The *Boomaga-IPP GUI Application* shall be designed to provide a simple and intuitive graphical interface to its user.*(L1-SYS-009)* It shall provide the primary functions enumerated in Table 3.2.*(L1-SYS-010)* All level 2 (*L2*) requirements for the *Boomaga-IPP* GUI are derived from the above requirements and shall be established and documented in the *Boomaga-IPP User Interface Specification (Martin & Boomaga-IPP Project Team, 2026-03-12/2026b)*.*(L1-SYS-011)*

Table 3.2.: *Boomaga-IPP GUI Application* Primary Functions.

Boomaga-IPP GUI Application shall:

1. Preview one or more files to be printed as it or they will appear coming out of the printer,
2. Rotate selected pages clockwise or counterclockwise, if desired,
3. Delete selected pages from a set of documents, if desired,
4. Add blank pages at selected locations to a set of documents, if desired,
5. Print the edited document file in half-page booklet format, if desired,
6. Print the edited document file with a user-selected number of pages per sheet, if booklet format is not desired,
7. Print the final output file to any suitable *CUPS* printer selected by the user.

(L1-SYS-010)

Boomaga-IPP Virtual Printer is designed to perform the functions of previewing, imposing (laying out), and printing a set of documents in a single GUI session. Consequently, its single GUI use case is *Preview, Impose (Lay Out), and Print a Set of Documents*. The textual form of the use case is shown as Table 3.3. The Unified Modeling Language (UML) use case diagram is shown as Figure 3.1. Additionally, the UML component diagram, class diagram, sequence diagram, and use case diagram for the *Boomaga-IPP Virtual Printer* system are included in Appendix C, Analysis Models, as Figures C.1, C.2, C.3, and C.4, respectively. The UML source code for all these diagrams are also in Appendix C as Tables C.1, *Boomaga-IPP Virtual Printer* System Component UML Model Source; C.2, *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 1; C.3, *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 2; C.4, *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 3; C.5, *Boomaga-IPP Virtual Printer* System Sequence UML Model Source; and C.6, *Boomaga-IPP Virtual Printer* Use Case UML Model Source.

3.2. Hardware Interfaces

The *Boomaga-IPP Virtual Printer* will have no direct interface to hardware. It shall, however, interface through the host computer system mouse, keyboard, and display to the user using *Wayland* protocol (Høgsberg & The Wayland Project, nodate), as described above.*(L1-SYS-012)* The Wayland interface to the *Boomaga-IPP* GUI application shall receive user requests from the host system and provide the output print job preview to the GUI application Job_Store_PDF_Preview feature.*(L1-SYS-013)* The *Boomaga-IPP Virtual Printer* shall also interface to any user-selected downstream physical printer hardware by using the host computer *CUPS* application programming interface (API) (Sweet & OpenPrinting, nodate) to send requests in the IPP protocol (Printer Working Group, 2024).*(L1-SYS-014)*

Table 3.3.: *Boomaga-IPP Virtual Printer* Use Case.

Preview, Impose (Lay Out), and Print a Set of Documents Goal Level: Sea Level
Main Success Scenario: 1. Print one or more documents to *Boomaga-IPP*. 2. Preview the resulting print job or jobs. 3. Manipulate Pages as required: a) Rotate pages as required. b) Delete pages as required. c) Add Blank Pages as required. 4. Select desired imposition (layout). 5. Select desired downstream printer. 6. Print job or jobs.
Extensions: 6a: Select desired Portable Document Format (PDF) output file: .1 Select "Print to File (PDF)" or "CUPS-PDF," (if CUPS-PDF is installed). .2 Select Filename and Folder from printer dialog.

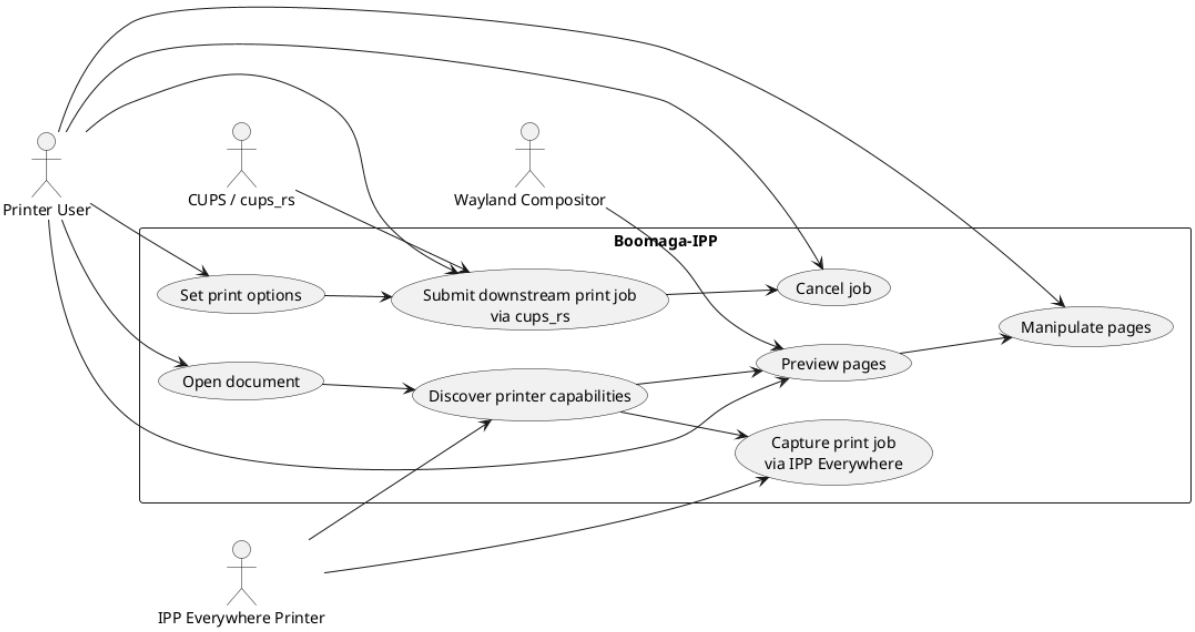

Figure 3.1.: *Boomaga-IPP Virtual Printer* Use Case.

3.3. Software Interfaces

3.3.1. Upstream Printer Interface

The *Boomaga-IPP* Capture_Daemon shall act as a virtual *IPP Everywhere™* printer available to user applications through the host computer *CUPS* server.*(L1-SYS-015)* The top level software interface requirements of the *Boomaga-IPP* Capture_Daemon are enumerated in Table 3.4.*(L1-SYS-016)*

Table 3.4.: *Boomaga-IPP* Capture_Daemon Software Interface Requirements

The *Boomaga-IPP* Capture_Daemon shall:

1. Expose an *IPP Everywhere™* print service (a driverless, locally-shared IPP print queue) to receive user print jobs forwarded through the host computer's CUPS server, allowing the user to print to the Boomaga-IPP Virtual Printer;
2. Advertise and accept, at minimum, the *IPP Everywhere™* mandatory document formats: PDF (application/pdf), PWG Raster (image/pwg-raster), and JPEG (image/jpeg) via its *IPP Everywhere™* print service;
3. Convert each accepted non-PDF document to PDF before adding it to the Imposition_Engine's Job_Store; and
4. Employ a ".socket" interface to allow it to be initiated when the user first prints to the *Boomaga-IPP Virtual Printer*.

(L1-SYS-016)

3.3.2. Interface to the User

The sole operational interface between the *Boomaga-IPP Virtual Printer* and its user shall be the *Boomaga-IPP GUI Application*.*(L1-SYS-017)* The top-level software interface requirements of the GUI_Application are enumerated in Table 3.5.*(L1-SYS-018)*

Table 3.5.: *Boomaga-IPP GUI Application* Software Interface Requirements

The *Boomaga-IPP GUI Application* shall:

1. Target Wayland (Høgsberg & The Wayland Project, nodate) display environments,
2. Employ the *Xilem GUI Framework* (Linebender Organization, 2022-11-16/2026),
3. Employ the *Poppler-rs* (Dröge et al., 2026) library for rendering PDF files in the GUI Job_Store_PDF_Preview feature, and
4. Comply with the requirements of the *Boomaga-IPP User Interface Specification* (Martin & Boomaga-IPP Project Team, 2026-03-12/2026b)

(L1-SYS-018)

3.3.3. Interface to the Downstream Printer

The interface between the *Boomaga-IPP Virtual Printer* and the downstream printer shall be provided by the *Boomaga-IPP* Downstream_Printer.*(L1-SYS-019)* The Downstream_Printer may act as an *IPP Everywhere™* client to submit the output print job directly to driverless downstream printers, falling back to the host *CUPS* server where direct IPP submission is unavailable.*(L1-SYS-070)* The top level software interface requirements of the *Boomaga-IPP* Downstream_Printer are enumerated in Table 3.6.*(L1-SYS-020)*

Table 3.6.: *Boomaga-IPP* Downstream_Printer Software Interface Requirements

The *Boomaga-IPP* Downstream_Printer shall:

1. Provide the *Boomaga-IPP Virtual Printer* access to the host computer CUPS server,
2. At the request of the *Boomaga-IPP GUI Application*, select a downstream printer and adjust its print settings, and
3. At the request of the *Boomaga-IPP GUI Application*, request the output print job from the Imposition_Engine and forward it to the *CUPS* server to be queued for printing on the selected downstream printer.

(L1-SYS-020)

3.4. Communications Interfaces

All *Boomaga-IPP Virtual Printer* IPC shall employ Unix domain sockets with a versioned JSON message protocol.*(L1-SYS-021)* Socket activation via *systemd* shall be used to start the Capture_Daemon on first use.*(L1-SYS-071)* IPC with *systemd* (service lifecycle/activation) shall use the *zbus_systemd* (Bruno & Khan, 2026) library.*(L1-SYS-072)* The data items to be exchanged by the *Boomaga-IPP Virtual Printer* software components/features are enumerated in Table 3.7.*(L1-SYS-022)*

Table 3.7.: *Boomaga-IPP* Communications Interface Requirements

The data items exchanged between the various components/features of the *Boomaga-IPP Virtual Printer* shall include:

1. Input print job(s) received by the Capture_Daemon (as specified in Subsection 3.3.1) shall be passed to the Imposition_Engine in PDF format,
2. Output print jobs in the form of PDF files shall be transmitted by the Imposition_Engine and received by the Downstream_Printer,
3. Requests to change the Imposition_Engine's state data describing the transformation of the input print job(s) to the output print job shall be sent by the GUI_Application in the form of messages and received by the Imposition_Engine,
4. The output print job preview shall be transmitted by the Imposition_Engine in the form of a PDF file and received by the GUI_Application,
5. The Downstream_Printer shall transmit information on downstream printers and printer settings in the form of messages and the messages shall be received by the GUI_Application,
6. The GUI_Application shall transmit requests to select a downstream printer and adjust its settings as desired by the user in the the form of messages and the messages shall be received by the Downstream_Printer,
7. The exchange of messages requesting the various actions necessary to perform the functions of the *Boomaga-IPP Virtual Printer* and the return of messages confirming the completion of those actions shall take place between the various software components/features of the *Boomaga-IPP Virtual Printer*.

(L1-SYS-022)

4. System Features

The *Boomaga-IPP Virtual Printer* software system architecture shall be modular in order to provide improved scalability, flexibility, and ease of maintenance; faster development cycles; and reduced code management complexity.*(L1-SYS-023)* The *Boomaga-IPP Virtual Printer* software system shall be composed of four primary features: three daemons (services) and the *Boomaga-IPP GUI Application*.*(L1-SYS-024)* The features shall be as enumerated in Table 4.1.

Table 4.1.: *Boomaga-IPP Virtual Printer* Primary System Features

The Primary System Features of the *Boomaga-IPP Virtual Printer* shall be:

1. The Capture_Daemon, that shall provide the *IPP Everywhere*™ client and other capabilities necessary to capture incoming print jobs and feed them to the Imposition_Engine's Job_Store;
2. The Imposition_Engine, that shall: 1) maintain the captured incoming print jobs, 2) maintain the state information necessary to transform the incoming jobs into the outgoing print job, 3) provide the print preview to the GUI_Application's Job_Store_PDF_Preview and, 4) on request, provide the output print job to the Downstream_Printer;
3. The GUI_Application, that shall be the direct interface between the *Boomaga-IPP Virtual Printer* software system and the user; and
4. The Downstream_Printer, that shall manage the interface between the GUI_Application, the Imposition_Engine, and the host computer's *CUPS* server and shall queue the output print job to the downstream printer on request.

(L1-SYS-024)

4.1. Capture_Daemon

4.1.1. Description and Priority

The Capture_Daemon shall act as the interface between the *Boomaga-IPP Virtual Printer* and the upstream printer.*(L1-SYS-025)* The Capture_Daemon shall present a driverless *IPP Everywhere*™ print service endpoint enabling the host *CUPS* server to queue incoming user print jobs to the *Boomaga-IPP Virtual Printer*.*(L1-SYS-026)* The Capture_Daemon shall feed incoming print jobs to the Imposition_Engine's Job_Store for later action as requested by the user.*(L1-SYS-027)* The Capture_Daemon shall be a High Priority service so as to assure prompt and timely response to any user printed incoming print job.*(L1-SYS-028)*

4.1.2. Stimulus/Response Sequences

The *Boomaga-IPP Virtual Printer IPP Everywhere™* client shall start the Capture_Daemon upon receiving an incoming print job if the Capture_Daemon is not already running.*(L1-SYS-029)* The *IPP Everywhere™* client shall then pass the queued incoming print job to the Capture_Daemon.*(L1-SYS-030)* The Capture_Daemon shall then start the Imposition_Engine if it is not already running.*(L1-SYS-031)* Once the Imposition_Engine is running, the Capture_Daemon shall pass the queued incoming print job to the Imposition_Engine.*(L1-SYS-032)* Once the Imposition_Engine has successfully received the queued incoming print job, the Capture_Daemon shall enter an idle state awaiting additional queued incoming print jobs from the *IPP Everywhere™* client.*(L1-SYS-033)*

4.1.3. Functional Requirements

The Capture_Daemon shall receive queued incoming print jobs through its *IPP Everywhere™* client.*(L1-SYS-034)* The Capture_Daemon shall immediately pass each received queued incoming print job to theImposition_Engine for addition to the Imposition_Engine's Job_Store.*(L1-SYS-035)*

4.2. Imposition_Engine

4.2.1. Description and Priority

The Imposition_Engine shall be the primary functional component of the *Boomaga-IPP Virtual Printer*.*(L1-SYS-036)* The Imposition_Engine shall receive queued incoming print jobs from the Capture_Daemon and add them to the Job_Store.*(L1-SYS-037)* The Imposition_Engine shall impose (lay out) the Job_Store in accordance with user requests received by it from the user through the GUI_Application.*(L1-SYS-038)* The Imposition_Engine shall provide the imposed Job_Store and related information to the *Boomaga-IPP GUI Application* to be displayed as a preview of the printed output to the user.*(L1-SYS-039)* The Imposition_Engine shall pass the imposed output of the Job_Store to the Downstream_Printer to be queued on the downstream printer upon user request.*(L1-SYS-040)* The Imposition_Engine shall be a Medium Priority service, lower in priority than the Capture_Daemon. *(L1-SYS-041)*

4.2.2. Stimulus/Response Sequences

The Imposition_Engine shall respond to requests from the Capture_Daemon by adding incoming print jobs to the Job_Store.*(L1-SYS-042)* The Imposition_Engine shall respond to requests from the user through the GUI_Application by changing the imposition state data defining the transformation of incoming print job(s) to the output print job.*(L1-SYS-043)* After changing the transformation state data, the Imposition_Engine shall provide the GUI_Application with an updated output print job preview and updated Job_Store status information for display to the user.*(L1-SYS-044)* The Imposition_Engine shall respond to requests from the Downstream_Printer and provide the imposed Job_Store for queueing on the downstream printer.*(L1-SYS-045)*

4.2.3. Functional Requirements

The Imposition_Engine shall conform to the functional requirements enumerated in Table 4.2.*(L1-SYS-046)*

Table 4.2.: *Boomaga-IPP* Imposition_Engine Functional Requirements

The Imposition_Engine shall:

1. Interface with the Capture_Daemon to receive one or more input print jobs and add them to the Job_Store;
2. Interface with the GUI_Application to provide the Job_Store_Contents_Listing feature with the information that it requires on input print jobs and sub-booklets;
3. Interface with the GUI_Application to provide the Job_Store_PDF_Preview feature with a preview of the current output print job;
4. Interace with the GUI_Application to receive user requests to change the Job_Store state data;
5. Immediately apply any user requests to change the Job_Store state data received from the GUI_Application to the Job_Store to ensure the correctness of the Job_Store_Contents_Listing, Job_Store_PDF_Preview, and the output print job PDF file;
6. Interface with the Downstream_Printer to provide it with the output print job PDF file for queueing to the downstream printer on request;[1] and
7. Compute page placement and imposition (N-up and booklet layouts, scaling, rotation, margins/gutter); and
8. Employ the *qpdf-rs* (Pankratov, 2026) library to assemble and apply content-preserving transforms to the resulting output PDF.

[1] The Grayscale Color mode shall be applied by the Imposition_Engine independently of the Downstream_Printer.

(L1-SYS-046)

4.3. GUI_Application

4.3.1. Description and Priority

The GUI_Application has been described in detail above in Sections 3.1.2, The *Boomaga-IPP* Graphical User Interface, and 3.3.2, Interface to the User. The GUI_Application shall be a Medium Priority service, lower in priority than the Capture_Daemon and equal in priority to the Imposition_Engine.*(L1-SYS-047)*

4.3.2. Stimulus/Response Sequences

The GUI_Application shall respond to the Imposition_Engine to receive Job_Store state data and to request changes to that state data.*(L1-SYS-048)* The GUI_Application shall also respond to the Imposition_Engine to receive a preview of the current Job_Store output print job PDF file.*(L1-SYS-049)* The GUI_Application shall interface with the Downstream_Printer so as to: 1) allow the user to select any *CUPS* printer as the downstream printer, 2) allow the user to set print options for the

selected printer, and 3) allow the user to request that the Job_Store output print job PDF file be queued to the selected printer.*(L1-SYS-050)*

4.3.3. Functional Requirements

The functional requirements of the *Boomaga-IPP GUI Application* are established in the *Boomaga-IPP User Interface Specification* (Martin & Boomaga-IPP Project Team, 2026-03-12/2026b).

4.4. Downstream_Printer

4.4.1. Description and Priority

The Downstream_Printer shall act as the interface between the *Boomaga-IPP Virtual Printer* and the user-selected downstream printer.*(L1-SYS-051)* The Downstream_Printer shall manage the *Boomaga-IPP Virtual Printer*'s interface with the host computer system's *CUPS* server so that the user may request that the imposed output print job be queued on the selected downstream printer.*(L1-SYS-052)* The Downstream_Printer shall be a Low Priority service, lower in priority than the Capture_Daemon, the Imposition_Engine, and the GUI_Application.*(L1-SYS-053)*

4.4.2. Stimulus/Response Sequences

The Downstream_Printer shall respond to requests from the *Boomaga-IPP GUI Application* for information on available *CUPS* printers and their settings.*(L1-SYS-054)* The Downstream_Printer shall respond to requests from the GUI_Application to select a downstream printer and change its settings.*(L1-SYS-055)* The Downstream_Printer shall respond to requests to queue the imposed output print job for printing on the selected downstream printer by relaying the imposed output print job from Imposition_Engine to the host computer's *CUPS* server.*(L1-SYS-056)*

4.4.3. Functional Requirements

All Downstream_Printer responses to the GUI_Application shall be complete and correct.*(L1-SYS-057)*

5. Other Nonfunctional Requirements

5.1. Performance Requirements

The *Boomaga-IPP GUI Application* shall respond to user requests within 1.0 seconds.*(L1-SYS-058)* The DOWNSTREAM_PRINTER shall respond to *Boomaga-IPP GUI Application* requests within 2.0 seconds.*(L1-SYS-059)* All other actions initated by the *Boomaga-IPP Virtual Printer* shall commence within 5.0 seconds and complete within 30 seconds.*(L1-SYS-060)*

5.2. Safety Requirements

5.2.1. Software Safety

The *Boomaga-IPP* Software Safety Guidelines (SSG) are provided in Appendix E. Any intentional deviation from the SSG shall be approved in advance by the *Boomaga-IPP* Configuration Control Board (CCB).*(L1-SYS-061)*

5.3. Security Requirements

The *Boomaga-IPP* Project Security Guidelines (PSG) are provided in Appendix F. Any intentional deviation from the PSG shall be approved in advance by the *Boomaga-IPP* CCB.*(L1-SYS-062)*

5.4. Software Quality Attributes

All *Boomaga-IPP Virtual Printer* software components shall be written in the *RUST*® programming language (Rust Foundation, nodate-c).*(L1-SYS-063)* The *Boomaga-IPP* Software Quality Guidelines (SQG) are provided in Appendix G. Any intentional deviation from the SQG shall be approved in advance by the *Boomaga-IPP* CCB.*(L1-SYS-064)*

5.5. Business Rules

The *Boomaga-IPP* Project shall operate as a free, open-source software project.*(L1-SYS-065)* All software available directly from its repository (github.com/GaryScottMartin/Boomaga-IPP) shall be freely available and released under the terms of the Free Software Foundation's GNU General Public License Version 3 (GPLv3), unless otherwise noted.*(L1-SYS-066)*

6. Other Requirements

Provisions for translation of the *Boomaga-IPP GUI Application* screens and *Boomaga-IPP Virtual Printer* installation instructions into languages other than English shall be made in order to make it possible for non-English speakers to use the *Boomaga-IPP Virtual Printer* at some later time.*(L1-SYS-069)*

Appendix A.

Glossary

Ada A modern programming language designed for large, long-lived applications – and embedded systems in particular – where reliability and efficiency are essential. It was most recently updated in 2022. Those revisions, know as Ada 2022, have been published as the International Standard ISO/IEC 8652:2023(E). For additional information, see Section 1.5. References: ISO/IEC JTC 1/SC 22/WG 9 - Ada, 2023.

Adobe® A registered trademark of Adobe, Inc..

ARCH LINUX™ *A simple, lightweight Linux® distribution™*. For additional information, see Section 1.5. References: Vinet et al., nodate.

Boomaga-IPP GUI Application The application software that provides the GUI between the user and the *Boomaga-IPP Virtual Printer*. For additional information, see the *Boomaga-IPP Project Repository*: Section 1.5. References: Martin and Boomaga-IPP Project Team, 2026-03-03/2026a.

Boomaga An orphaned virtual *CUPS* printer system for *Linux®* computers. The last release, v3.0.0, was issued Mar 13, 2019. A patch to the v3.0.0 source code was merged into the *Boomaga* source code on February 20, 2022 with no subsequent release. *Boomaga* has been unmaintained since that date. For additional information, see Section 1.5. References: Sokolov et al., 2013b.

Boomaga-IPP An *IPP Everywhere™* / *CUPS* virtual printer development project for *Linux®* computers. For additional information, see the *Boomaga-IPP Project Repository*: Section 1.5. References: Martin and Boomaga-IPP Project Team, 2026-03-03/2026a.

Boomaga-IPP Project Repository The *Boomaga-IPP* Project Repository for software and documentation, currently a public repository on *GitHub®*. For additional information, see the *Boomaga-IPP* Project Respoitory: Section 1.5. References: Martin and Boomaga-IPP Project Team, 2026-03-03/2026a.

Boomaga-IPP Virtual Printer An *IPP Everywhere™* / *CUPS* virtual printer software system for *Linux®* computers using *Wayland* display environments and managed via *systemd*; written entirely in *RUST®*. For additional information, see the *Boomaga-IPP Project Repository*: Section 1.5. References: Martin and Boomaga-IPP Project Team, 2026-03-03/2026a.

Capture_Daemon The *Boomaga-IPP* client/server software component responsible for receiving (capturing) input print jobs as PDF files through an *IPP Everywhere™* print service and adding them to the Job_Store..

Cargo® The official package manager and build system for the *RUST®* programming language. For additional information, see Section 1.5. References: Rust Foundation, nodate-a.

CLAUDE® A series of LLMs developed by Anthropic PBC and first released in 2023. *CLAUDE®* is used for software development via *CLAUDE® Code*. For additional information, see Section 1.5. References: Anthropic PBC, nodate-a.

CLAUDE® Code An agentic AI coding tool developed by Anthropic PBC that reads entire codebases, edits files, runs commands, and manages git workflows through natural language prompts. It operates autonomously across multiple files to execute tasks such as building features, fixing bugs, and refactoring, while requiring explicit user permission for file modifications and command execution. For additional information, see Section 1.5. References: Anthropic PBC, nodate-b.

CMake A free, cross-platform, software development tool for building applications via compiler-independent instructions. It also can automate testing, packaging and installation. It runs on a variety of platforms and supports many programming languages. For additional information, see 1.5. References: Cedilnik et al., 2026.

crates.io The *RUST®* community's shared software crate registry. For additional information, see Section 1.5. References: Rust Foundation, nodate-b.

CUPS A modular printing system for Unix-like computer operating systems which allows a computer to act as a print server. Formerly an acronym for Common UNIX Printing System. For additional information, see Section 1.5. References: Sweet and OpenPrinting, nodate.

D-Bus A desktop message bus system, a simple way for applications to talk to one another. *D-Bus* provides interprocess communication and helps coordinate process lifecycle; it makes it simple and reliable to code a "single instance" application or daemon, and to launch applications and daemons on demand when their services are needed. For additional information, see Section 1.5. References: Pennington et al., nodate.

document page A single page of a source PDF document (an input print job) captured by the *Boomaga-IPP* Capture_Daemon. Equivalent to a logical page.

Downstream_Printer The downstream physical or virtual printer to which the *Boomaga-IPP Virtual Printer* submits its imposed Job_Store through the host computer's *CUPS* server.

Druid A data-first, *RUST®*-native *UI* design toolkit designed to offer a polished user experience for cross-platform desktop applications. While the project is no longer under active development by the *Linebender Organization*, version 0.8.3 remains available on *crates.io* and on *GitHub®* for bug fixes and documentation improvements. For additional information, see Section 1.5. References: Levien et al., 2023.

GhostScript® A suite of software based on an interpreter for *Adobe®*'s *POSTSCRIPT®* and PDF page description languages. Its main purposes are the rasterization of documents in these languages, the display or printing of document pages, and conversion between *POSTSCRIPT®* and PDF files. For additional information, see Section 1.5. References: Artifex Software, Inc., nodate.

git A free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. For additional information, see Section 1.5. References: Torvalds et al., nodate.

GitHub® A proprietary developer platform that allows developers to create, store, manage, and share their code. It uses git to provide distributed version control and *GitHub* itself provides access control, bug tracking, software feature requests, task management, continuous integration, and wikis for every project. For additional information, see Section 1.5. References: GitHub, Inc. (A subsidiary of Microsoft Corporation), 2026.

GNU Make A tool that controls the generation of executables and other non-source files of a program from the program's source files. For additional information, see 1.5. References: GNU Project and Smith, 2026.

GUI_Application The application software that provides the GUI between the user and the *Boomaga-IPP Virtual Printer*. For additional information, see Section 1.5. References: Martin and Boomaga-IPP Project Team, 2026-03-03/2026a.

impose To arrange one or more document pages (or reader spreads) onto a set of facing sheets in a specified orientation and order (referred to as a printer spread) so as to produce a desired output from a printer.

imposed Arranged onto a set of facing pages (a printer spread) in a specified orientation and order so as to produce a desired output from a printer.

imposed page A single page of a PDF output document (a print job) arranged and oriented as specified by the *Boomaga-IPP* user and ready to be submitted to the *Boomaga-IPP* Downstream_Printer. A set of facing imposed pages is referred to as a printer spread.

imposition The process of arranging and orienting (imposing) document pages onto printer spreads for printing.

Imposition_Engine The *Boomaga-IPP* client/server software component responsible for managing the Job_Store and its associated state information. The Imposition_Engine serves the imposed Job_Store to the *Boomaga-IPP GUI Application* Job_Store_PDF_Preview feature and to the *Boomaga-IPP* Downstream_Printer on demand.

IPP Everywhere™ An extension of IPP to support network printing without vendor-specific driver software, including the transport, various discovery protocols, and standard document formats. For additional information, see Section 1.5. References: Printer Working Group, 2020.

Job_Store The *Boomaga-IPP* software component containing the Job_Store data structure and related access methods. Print jobs received through the *IPP Everywhere™* client are added to the Job_Store. The *Boomaga-IPP* GUI accesses and modifies the user-commanded output state of the Job_Store. The *Boomaga-IPP* Downstream_Printer requests the imposed Job_Store and relays it to the user-selected downstream printer on request.

Job_Store_Contents_Listing The JOB_STORE_CONTENTS_LISTING feature of the *BOOMAGA-IPP* GUI (*UI* FEATURE 1).

Job_Store_PDF_Preview The JOB_STORE_PDF_PREVIEW feature of the *Boomaga-IPP GUI Application* (*UI* Feature 2).

KDE An international community developing high-quality, free, and open-source software. KDE was originally an acronym for the K Desktop Environment, a wordplay on the pre-existing *Common Desktop Environment*. For additional information, see Section 1.5. References: KDE Community, 2026.

L1 Designates a Level 1 requirement within a requirement ID of the form LX-SS-nnn. LX would be L1 for Level 1 Requirements, L2 for Level 2, etc. SS is the subsystem: SYS for System Wide, UI for User Interface, etc.

L2 Designates a Level 2 requirement within a requirement ID of the form LX-SS-nnn. LX would be L1 for Level 1 Requirements, L2 for Level 2, etc. SS is the subsystem: SYS for System Wide, UI for User Interface, etc.

Linebender Organization "A friendly group of people who share an interest in 2D graphics and user interface design, with everything that entails." Most *Linebender* projects are developed in the Rust programming language. For additional information, see Section 1.5. References: Linebender Organization, nodate-a.

Linux® A very large family of *UNIX®*-like operating systems based on a kernel first produced by Linus Torvalds in 1991. For additional information, see Section 1.5. References: Linus Torvalds et al., nodate and Linux Foundation, nodate.

LLAMA® A family of AI models produced by *Meta AI*, a division of *Meta Platforms, Inc*. For additional information, see Section 1.5. References: Meta AI, nodate.

logical page A single page of a source PDF document (an input print job) captured by the *Boomaga-IPP* CAPTURE_DAEMON. Equivalent to a document page.

opencode® An open source AI coding agent produced by Anomaly Innovations Inc. For additional information, see Section 1.5. References: Anomaly Innovations Inc., nodate.

page One side of a sheet of paper. Within this document, logical page or document page refers to a single page of a source PDF document (a print job) captured by *Boomaga-IPP* CAPTURE_DAEMON. Imposed page refers to a single page imposed for submission to the *Boomaga-IPP* DOWNSTREAM_PRINTER.

PDF A file format developed by Adobe in 1993 and standardized as ISO 32000 in 2008. The PDF format is used to present documents, including text formatting and images, in a manner independent of application software, hardware, and operating systems. Based on the PostScript language, a PDF file encapsulates a complete description of a fixed-layout document, including the text, fonts, vector graphics, raster images and other information needed to display it. For additional information, see 1.5. References: ISO/TC 171/SC 2, 2008.

Perplexity.ai A worldwide web based tool for querying various online AI models, primarily its own search engine, known as Sonar, based on a Meta *LLAMA*® model. A product of *Perplexity AI, Inc*. For additional information, see Section 1.5. References: Srinivas et al., nodate.

PlantUML An open-source tool allowing users to create diagrams from a plain text language. Besides various UML diagrams, PlantUML has support for various other software development related formats (such as Archimate, Block diagram, BPMN, C4, Computer network diagram, ERD, Gantt chart, Mind map, and WBD), as well as visualisation of JSON and YAML files. For additional information, see Section 1.5. References: Roques and PlantUML Contributors, 2026.

Plasma KDE's desktop environment. For additional information, see Section 1.5. References: KDE Community, nodate.

Poppler a PDF rendering library based on the xpdf-3.0 code base. For additional information, see Section 1.5. References: Høgsberg and The Poppler Project, nodate.

Poppler-rs A high-level (safe) set of Rust bindings for *Poppler*'s glib interface. *Poppler* is a PDF rendering library with a *cairo* backend. For additional information, see Section 1.5. References: Dröge et al., 2026.

POSTSCRIPT® A page description language and dynamically typed, stack-based programming language. For additional information, see Section 1.5. References: Adobe, Inc., nodate.

printer spread An imposed spread ready for submission to a printer.

qpdf A command-line tool and C++ library that performs content-preserving transformations on PDF files. It supports linearization, encryption, and numerous other features. It can also be used for splitting and merging files, creating PDF files (but you have to supply all the content yourself), and inspecting files for study or analysis. For additional information, see Section 1.5. References: Berkenbilt and Holger, nodate.

qpdf-rs Rust safe bindings to the QPDF C++ library. It uses the QPDF C API exposed via qpdf-c.h header. For additional information, see Section 1.5. References: Berkenbilt and Holger, nodate.

reader spread A spread of a source PDF as received by the *Boomaga-IPP* Capture_Daemon/*IPP Everywhere*™ client for imposition and printing.

RUST® A general-purpose programming language noted for its emphasis on performance, type safety, concurrency, and memory safety. For additional information, see Section 1.5. References: Rust Foundation, nodate-c.

spread A set of facing pages in a book or booklet..

SYS A subsystem identifier in the *Boomaga-IPP* requirements numbering scheme used to identify system level (*L1*) requirements..

systemd A software suite for system and service management on *Linux*® built to unify service configuration and behavior across *Linux*® distributions. For additional information, see Section 1.5. References: Poettering and Sievers, nodate.

UI A subsystem identifier in the *Boomaga-IPP* requirements numbering scheme; used to identify user interface requirements. Also an acronym for user interface.

UNIX® A family of multi-tasking, multi-user computer operating systems that derive from the original AT&T Unix, the development of which started in 1969 at the Bell Labs research center by Ken Thompson, Dennis Ritchie, and others. For additional information, see Section 1.5. References: The Open Group, nodate.

Wayland A replacement for the X11 window system protocol and architecture with the aim to be easier to develop, extend, and maintain. *Wayland* is the language (protocol) that applications can use to talk to a display server in order to make themselves visible and get input from the user (a person). A *Wayland* server is called a "compositor." Applications are *Wayland* clients. For additional information, see see Section 1.5. References: Høgsberg and The Wayland Project, nodate.

X11 A windowing system for bitmap displays, common on Unix-like operating systems. X originated as part of Project Athena at Massachusetts Institute of Technology (MIT) in 1984. The X protocol has been at version 11 (hence "X11") since September 1987. For additional information, see Section 1.5. References: Gettys et al., nodate.

Xilem An experimental, high-performance *RUST® GUI Framework* under development by the Linebender community, designed to create cross-platform desktop and mobile applications. Inspired by *SwiftUI*, *Flutter*, and *Elm*, it utilizes a reactive architecture. For additional information, see Section 1.5. References: Linebender Organization, 2022-11-16/2026 and Linebender Organization, nodate-b.

zbus_systemd A *RUST®* library for communication with *systemd* processes. For additional information, see Section 1.5. References: Bruno and Khan, 2026.

Appendix B.

Acronyms

AI artificial intelligence.

API application programming interface.

CCB Configuration Control Board.

FAQ frequently asked questions.

GUI graphical user interface.

IEEE Institute of Electrical and Electronic Engineers.

IPC inter-process communication.

IPP Internet Printing Protocol.

ISTO Industry Standards and Technology Organization.

LLM large language model.

PDF Portable Document Format.

PSG Project Security Guidelines.

PWG Printer Working Group.

SQG Software Quality Guidelines.

SRS software requirements specification.

SSG Software Safety Guidelines.

TBD to be determined.

UIS user interface specification.

UML Unified Modeling Language.

Appendix C.

Analysis Models

Perplexity.ai was used to produce the following UML models based on the synthesis of the *Boomaga* virtual printer as currently implemented and the initial *Boomaga-IPP* top level requirements. *PlantUML* was then used to generate UML diagrams from the models.

Component Diagram

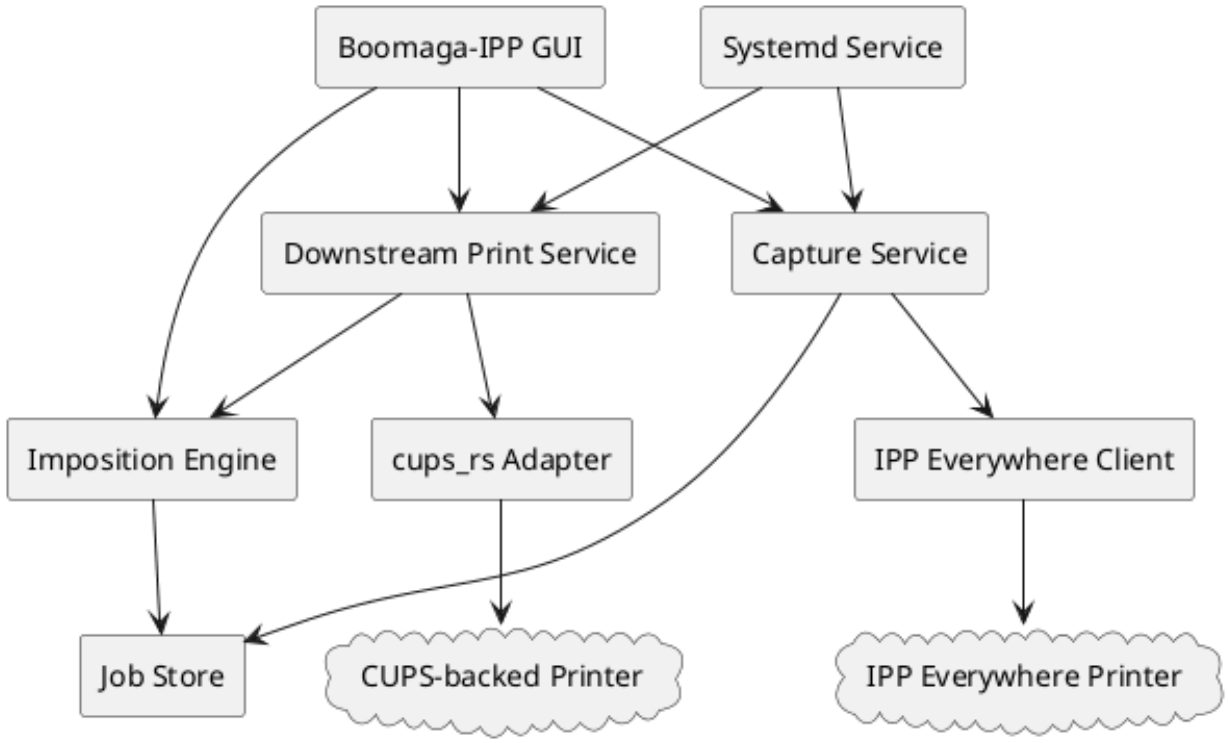

Figure C.1.: *Boomaga-IPP Virtual Printer* System Component Diagram.

Table C.1.: *Boomaga-IPP Virtual Printer* System Component UML Model Source.

```
@startuml
skinparam componentStyle rectangle

component "Boomaga-IPP GUI" as GUI
component "Capture Service" as CAP
component "IPP Everywhere Client" as IPP
component "Imposition Engine" as IMP
component "Job Store" as STORE
component "Downstream Print Service" as
 ↪  DPS
component "cups_rs Adapter" as CUPS
component "Systemd Service" as SD

cloud "IPP Everywhere Printer" as SOURCE
cloud "CUPS-backed Printer" as TARGET

GUI --> CAP
GUI --> IMP
GUI --> DPS
CAP --> IPP
CAP --> STORE
IMP --> STORE
DPS --> IMP
DPS --> CUPS
SD --> CAP
SD --> DPS

IPP --> SOURCE
CUPS --> TARGET
@enduml
```

Class Diagram

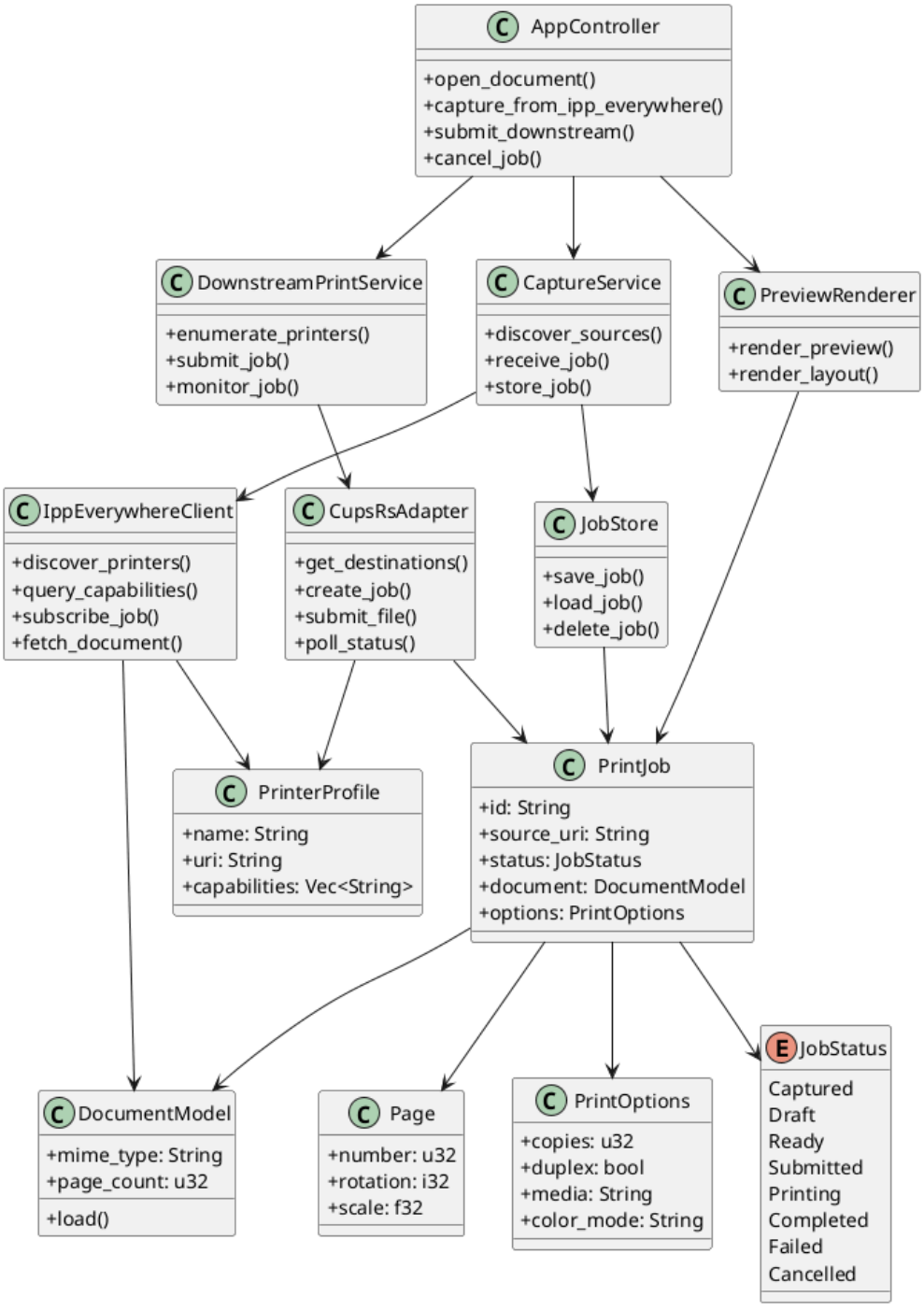

Figure C.2.: *Boomaga-IPP Virtual Printer* System Class Diagram.

Table C.2.: *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 1.

```
@startuml
skinparam classAttributeIconSize 0

class AppController {
  +open_document()
  +capture_from_ipp_everywhere()
  +submit_downstream()
  +cancel_job()
}

class CaptureService {
  +discover_sources()
  +receive_job()
  +store_job()
}

class IppEverywhereClient {
  +discover_printers()
  +query_capabilities()
  +subscribe_job()
  +fetch_document()
}

class JobStore {
  +save_job()
  +load_job()
  +delete_job()
}

class PrintJob {
  +id: String
  +source_uri: String
  +status: JobStatus
  +document: DocumentModel
  +options: PrintOptions
}

class DocumentModel {
  +mime_type: String
  +page_count: u32
  +load()
}
```

Table C.3.: *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 2.

```
class Page {
  +number: u32
  +rotation: i32
  +scale: f32
}

class PreviewRenderer {
  +render_preview()
  +render_layout()
}

class
 ↪ DownstreamPrintService
 ↪ {
  +enumerate_printers()
  +submit_job()
  +monitor_job()
}

class CupsRsAdapter {
  +get_destinations()
  +create_job()
  +submit_file()
  +poll_status()
}

class PrinterProfile {
  +name: String
  +uri: String
  +capabilities:
   ↪ Vec<String>
}

class PrintOptions {
  +copies: u32
  +duplex: bool
  +media: String
  +color_mode: String
}
```

Table C.4.: *Boomaga-IPP Virtual Printer* System Class UML Model Source, Page 3.

```
enum JobStatus {
  Captured
  Draft
  Ready
  Submitted
  Printing
  Completed
  Failed
  Cancelled
}

AppController --> CaptureService
AppController --> PreviewRenderer
AppController --> DownstreamPrintService
CaptureService --> IppEverywhereClient
CaptureService --> JobStore
IppEverywhereClient --> PrinterProfile
IppEverywhereClient --> DocumentModel
JobStore --> PrintJob
PrintJob --> DocumentModel
PrintJob --> Page
PrintJob --> PrintOptions
PreviewRenderer --> PrintJob
DownstreamPrintService --> CupsRsAdapter
CupsRsAdapter --> PrinterProfile
CupsRsAdapter --> PrintJob
PrintJob --> JobStatus
@enduml
```

Sequence Diagram

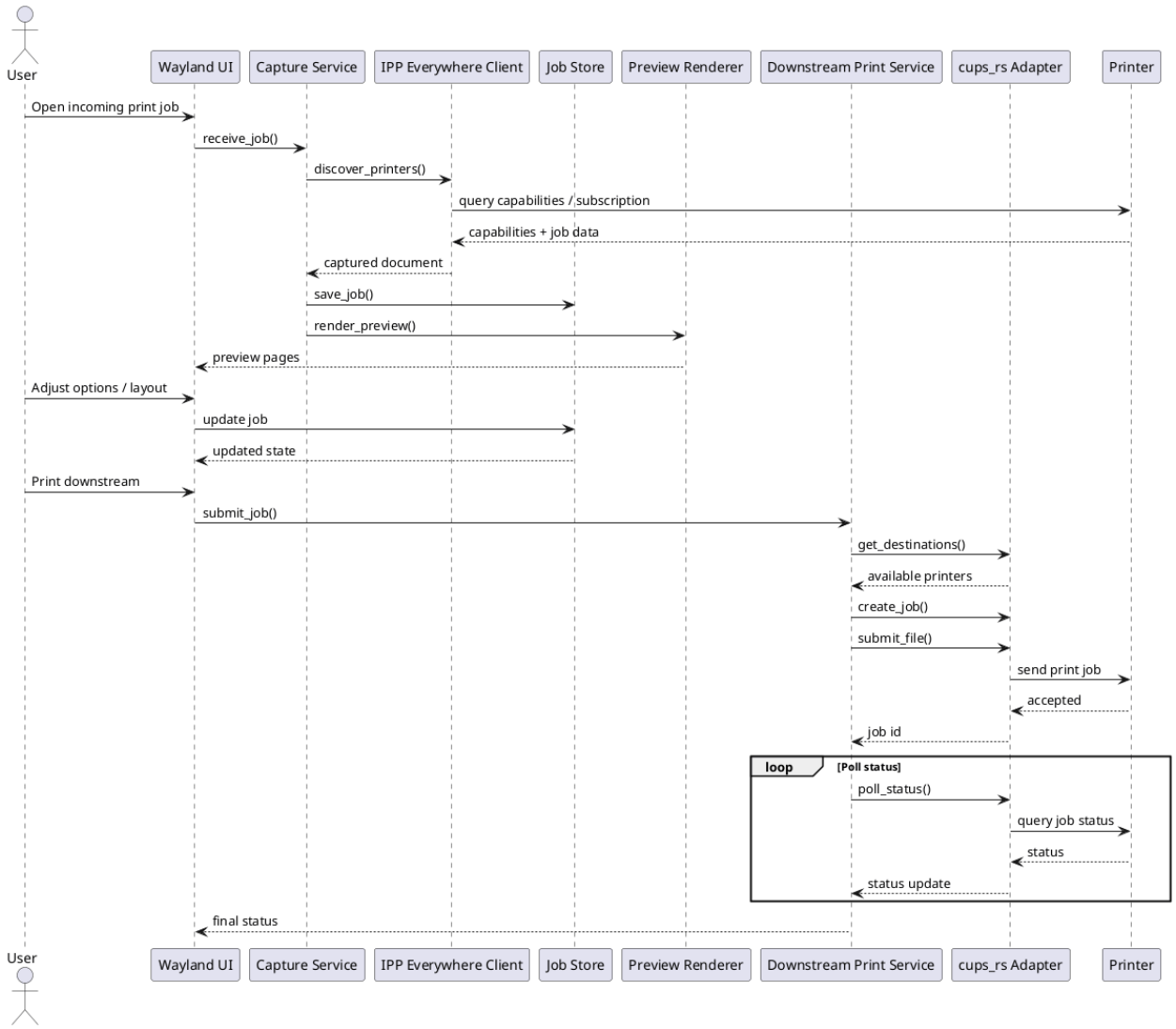

Figure C.3.: *Boomaga-IPP Virtual Printer* System Sequence Diagram.

Table C.5.: *Boomaga-IPP Virtual Printer* System Sequence UML Model Source.

```
@startuml
actor User
participant "Wayland UI" as UI
participant "Capture Service" as CS
participant "IPP Everywhere Client" as IPP
participant "Job Store" as STORE
participant "Preview Renderer" as PR
participant "Downstream Print Service" as DPS
participant "cups_rs Adapter" as CUPS
participant "Printer" as P

User -> UI : Open incoming print job
UI -> CS : receive_job()
CS -> IPP : discover_printers()
IPP -> P : query capabilities / subscription
P --> IPP : capabilities + job data
IPP --> CS : captured document
CS -> STORE : save_job()
CS -> PR : render_preview()
PR --> UI : preview pages

User -> UI : Adjust options / layout
UI -> STORE : update job
STORE --> UI : updated state

User -> UI : Print downstream
UI -> DPS : submit_job()
DPS -> CUPS : get_destinations()
CUPS --> DPS : available printers
DPS -> CUPS : create_job()
DPS -> CUPS : submit_file()
CUPS -> P : send print job
P --> CUPS : accepted
CUPS --> DPS : job id

loop Poll status
  DPS -> CUPS : poll_status()
  CUPS -> P : query job status
  P --> CUPS : status
  CUPS --> DPS : status update
end

DPS --> UI : final status
@enduml
```

Use Case Diagram

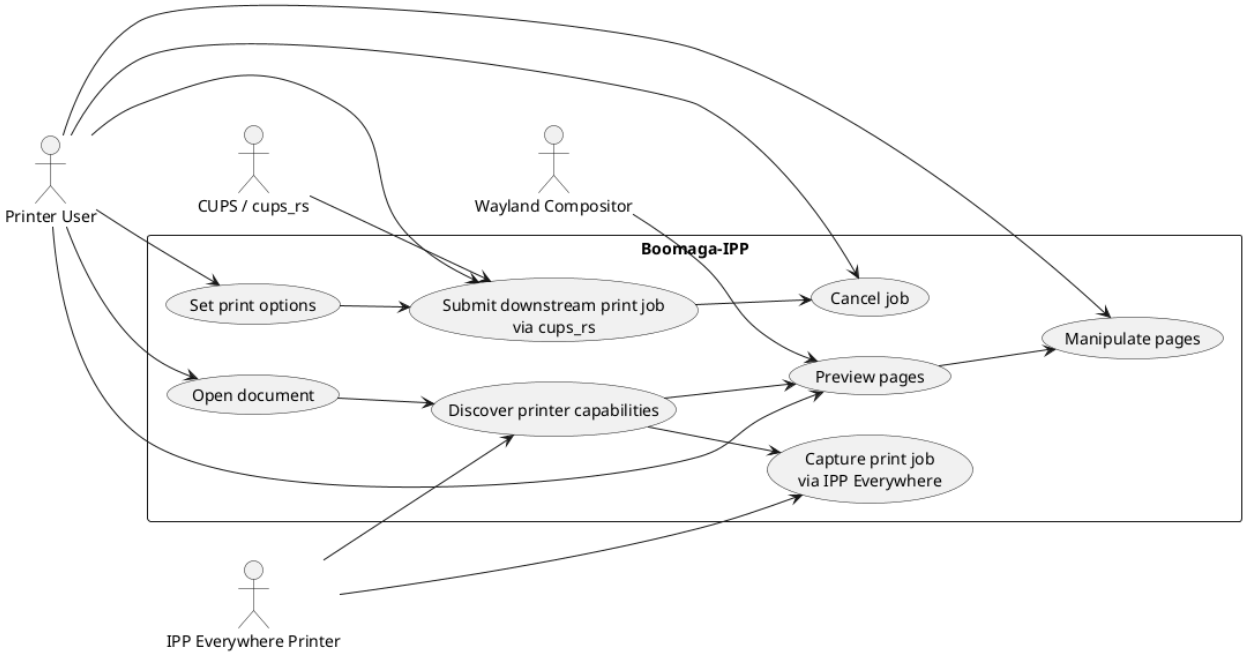

Figure C.4.: *Boomaga-IPP Virtual Printer* Use Case Diagram.

Table C.6.: *Boomaga-IPP Virtual Printer* Use Case UML Model Source.

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Printer User" as User
actor "IPP Everywhere Printer" as Printer
actor "Wayland Compositor" as Wayland
actor "CUPS / cups_rs" as CUPS

rectangle "Boomaga-IPP" {
  usecase "Open document" as UC1
  usecase "Discover printer capabilities" as UC2
  usecase "Capture print job\nvia IPP Everywhere" as UC3
  usecase "Preview pages" as UC4
  usecase "Manipulate pages" as UC5
  usecase "Set print options" as UC6
  usecase "Submit downstream print job\nvia cups_rs" as UC7
  usecase "Cancel job" as UC8
}

User --> UC1
User --> UC4
User --> UC5
User --> UC6
User --> UC7
User --> UC8

Printer --> UC2
Printer --> UC3
Wayland --> UC4
CUPS --> UC7

UC1 --> UC2
UC2 --> UC3
UC2 --> UC4
UC4 --> UC5
UC6 --> UC7
UC7 --> UC8
@enduml
```

Appendix D.

To Be Determined (TBD) List

No System Level TBDs are currently defined.

Appendix E.

Software Safety Guidelines

Core Requirements

1. Treat all external input as untrusted, including print jobs, PDFs, IPP requests, filenames, metadata, and environment variables.
2. Validate and reject malformed or oversized input early.
3. Run the service with least privilege, using a dedicated non-login user and minimal filesystem/network access.
4. Keep secrets out of the repository and out of logs.
5. Make failures safe by default: reject, quarantine, or sandbox suspicious jobs rather than processing them normally.
6. Log security-relevant events without exposing sensitive data.

File and Job Handling

1. Store uploaded or generated job files in a private, access-controlled directory.
2. Use per-job isolation so one print job cannot read or overwrite another.
3. Enforce strict path normalization to block path traversal and symlink attacks.
4. Limit file sizes, page counts, image dimensions, attachment counts, and decompression ratios.
5. Reject embedded active content unless it is explicitly required and sandboxed.

Parsing and Rendering

1. Use memory-safe libraries where possible.
2. Pin and review parser dependencies, especially for PDF, PostScript, image, and archive handling.
3. Isolate renderers in separate processes or containers with seccomp/AppArmor/SELinux-style confinement.
4. Impose timeouts and resource limits on parsing, rendering, and conversion steps.
5. Assume documents can be crafted to trigger crashes, hangs, or denial of service.

Network and Protocol

1. Authenticate administrative actions and device-management endpoints.
2. Require TLS for any remote management or IPP-over-network operation that crosses a trust boundary.
3. Rate-limit requests and enforce connection limits.
4. Validate IPP attributes and reject unexpected combinations rather than passing them downstream.

Secrets and Configuration

1. Never hardcode passwords, API keys, tokens, or private certificates.
2. Load secrets from protected runtime configuration, not source files.
3. Support secret rotation without code changes.
4. Redact secrets from logs, crash dumps, and diagnostics.
5. Avoid shipping default credentials.

Sandboxing and Privilege Separation

1. Separate the web/API layer, queue manager, and render/print worker into distinct processes.
2. Give each component only the permissions it needs.
3. Prevent the worker from writing outside its spool directory.
4. Consider container or VM isolation for the rendering path.

Availability and Abuse Resistance

1. Apply quotas per user or per job source.
2. Detect runaway jobs and kill them cleanly.
3. Make queue state durable and recoverable after crashes.
4. Prevent a single malformed job from taking down the whole service.
5. Add backpressure so the system degrades gracefully under load.

Build and Supply Chain

1. Use reproducible builds where practical.
2. Pin dependency versions and audit updates.
3. Scan third-party dependencies for known vulnerabilities.
4. Sign release artifacts and verify them in deployment.
5. Protect CI/CD secrets and restrict who can publish releases.

Operational Safety

1. Provide a secure-by-default configuration profile.
2. Separate debug mode from production mode.
3. Keep audit logs for admin actions and job failures.
4. Add a secure update mechanism.
5. Document how to recover from compromised queues, config, or secrets.

Good Acceptance Criteria

1. A malicious print job cannot execute arbitrary code in the main service process.
2. A malformed job cannot read arbitrary local files.
3. A queue user cannot access another user's jobs.
4. A denial-of-service attempt is bounded by quotas and timeouts.
5. No secrets are present in public repositories or client-visible config.

Appendix F.

Project Security Guidelines

Identity and Access

1. All contributors with write access **MUST** use multi-factor authentication.
2. Access **MUST** follow least privilege.
3. Administrative access **MUST** be limited to the minimum necessary set of maintainers.
4. Access **MUST** be revoked immediately when no longer required.

Code Change Control

1. All changes **MUST** enter through pull requests.
2. The default branch **MUST** be protected.
3. Protected branches **MUST** require at least one approved review.
4. Protected branches **MUST** require passing status checks before merge.
5. Force-pushes and branch deletion on protected branches **MUST NOT** be allowed.

Secrets

1. Secrets **MUST NOT** be committed to the repository.
2. Secrets **MUST** be stored only in approved secret-management systems.
3. Secret scanning **MUST** be enabled where supported.
4. Exposed secrets **MUST** be rotated immediately.
5. Secrets **SHOULD NOT** appear in logs, issue trackers, or screenshots.

Dependencies and Supply Chain

1. Dependencies **SHOULD** be pinned or locked where practical.
2. Third-party dependencies **MUST** be reviewed before adoption.
3. Known vulnerable dependencies **MUST NOT** remain in use when a fix is available.
4. Third-party build actions, packages, and plugins **SHOULD** come only from trusted sources.

CI/CD

1. CI/CD runners **MUST** use least privilege.
2. Deployment credentials **MUST** be short-lived where possible.
3. Secrets **MUST NOT** be exposed to untrusted pull requests.
4. Workflow changes **MUST** be reviewed with the same rigor as application code.
5. Build, test, and deploy privileges **SHOULD** be separated.

Repository Hygiene

1. The repository **SHOULD** include a security policy.
2. The repository **SHOULD** include contribution and review guidelines.
3. Generated files, local secrets, and machine-specific artifacts **MUST NOT** be committed.
4. `.gitignore` **MUST** cover secrets and build outputs.
5. Repository history **SHOULD** be audited periodically for leaked secrets.

Logging and Monitoring

1. Security-relevant administrative actions **MUST** be logged where supported.
2. Logs **MUST NOT** contain secrets.
3. Unusual access, permission changes, and workflow edits **SHOULD** be monitored.

Incident Readiness

1. The project **MUST** document how to respond to leaked secrets or compromised accounts.
2. The project **MUST** define who can revoke access and rotate credentials.
3. Recovery from malicious commits, accidental force-pushes, and compromised releases **SHOULD** be documented.
4. Backups of critical repository data **SHOULD** be maintained.

Minimum Baseline

1. MFA **MUST** be enforced.
2. Pull-request review **MUST** be required.
3. Default branch protection **MUST** be enabled.
4. Secret scanning **MUST** be enabled where available.
5. Secrets **MUST NOT** be stored in the repository.
6. Access revocation **MUST** be prompt.

Appendix G.

Software Quality Guidelines

Functional Correctness

1. Correctly implement IPP and CUPS protocols according to their specifications.
2. Correctly handle all supported document formats (PDF, PostScript, images) and preserve fidelity.
3. Correctly implement print-preview, booklet lay out, page ordering, scaling, and cropping.
4. Ensure that options (paper size, resolution, duplex, tray, color mode) are faithfully translated into CUPS/IPP commands.
5. Test edge cases: empty files, large documents, malformed PDFs, and extreme page counts.

Reliability and Robustness

1. Never crash on malformed input; fail gracefully with clear error messages.
2. Handle resource exhaustion (disk, memory, file descriptors) without corrupting state.
3. Make queue state and job metadata durable and recoverable after crashes or restarts.
4. Avoid deadlocks and livelocks in queue processing and worker threads.
5. Implement timeouts and cancellation for long-running jobs.

Performance

1. Keep startup time and per-job latency low for typical workloads.
2. Avoid unnecessary CPU work per page; optimize rendering and layout paths.
3. Handle large jobs without excessive memory usage; stream where possible.
4. Avoid blocking the UI thread during heavy processing.
5. Provide measurable performance budgets for common operations (e.g., “preview 100 pages in under X seconds”).

Security

1. Treat all input as untrusted: files, filenames, metadata, and IPP attributes.
2. Sandbox parsers and renderers to isolate potential exploits.
3. Never execute arbitrary code from print jobs or user input.
4. Store job files and spool data in private, access-controlled locations.
5. Implement strict path normalization and prevent path traversal.
6. Enforce quotas and rate limits to prevent denial-of-service.

Maintainability

1. Use a clear module boundary between protocol handling, document parsing, layout and rendering, queue management, and UI.
2. Keep functions small, focused, and testable.
3. Avoid global mutable state; use predictable configuration and dependency injection where possible.
4. Document public APIs and internal interfaces.
5. Keep build and dependency versions pinned and reproducible.

Testability

1. Provide a comprehensive unit-test suite for parsing, layout, and protocol logic.
2. Provide integration tests that exercise end-to-end print jobs, including malformed inputs.
3. Provide property-based or fuzz tests for parsers and document renderers.
4. Provide regression tests for known problematic documents.
5. Ensure tests are deterministic and fast enough for frequent CI runs.

Portability and Compatibility

1. Support the target operating systems as claimed (e.g., major Linux distributions, Windows, macOS).
2. Support standard CUPS backends and IPP versions.
3. Avoid non-portable system calls or hard-coded paths; use standard library abstractions.
4. Support common locales and Unicode in filenames and document metadata.
5. Be tolerant of different CUPS configurations and system layouts.

User Experience and Usability

1. Provide clear, actionable error messages for failed jobs and unsupported options.
2. Provide sensible defaults for common print configurations.
3. Offer sane fallback behavior when an option is unsupported.
4. Provide progress feedback for long-running jobs (e.g., progress bars, logs).
5. Provide a stable, documented CLI or API for scripting and automation.

Configuration and Documentation

1. Provide clear documentation for installation and dependencies.
2. Provide clear documentation for configuration options.
3. Provide clear documentation for command-line usage.
4. Provide clear documentation for troubleshooting.
5. Document supported features and known limitations.
6. Provide examples for common use cases (booklets, duplex, stapling, etc.).
7. Provide a CHANGELOG with meaningful summaries of changes.

Code Quality and Style

1. Use consistent coding style and formatting.
2. Enforce style with automated tools (linters, formatters).
3. Avoid unresolved warnings from the compiler and linters.
4. Use meaningful names for functions, types, and variables.
5. Keep cyclomatic complexity low in critical paths.

Versioning and Release Quality

1. Follow semantic versioning or a clear versioning scheme.
2. Ensure backward compatibility for minor versions where feasible.
3. Provide clear upgrade notes and migration guidance.
4. Sign release artifacts and provide checksums.
5. Avoid breaking changes in patch versions.

Observability and Debugging

1. Provide structured logging with appropriate log levels.
2. Avoid logging sensitive information (user data, full documents).
3. Provide diagnostic modes and verbose flags for debugging.
4. Provide clear error codes and categories for programmatic handling.
5. Include version and configuration information in diagnostics.

Accessibility and Inclusivity

1. Support Unicode in all user-facing strings and file paths.
2. Avoid hard-coded locale assumptions.
3. Provide clear, non-technical language where possible in user-facing messages.
4. Design CLI and UI to be scriptable and automatable.

Governance and Process Quality

1. Define a contribution and review process.
2. Require code review for all changes.
3. Use CI to run tests, linting, and security checks on every change.
4. Define what constitutes “done” for a feature or bug fix.
5. Track and prioritize technical debt.